ИНФОРМАТИКА · ЛЕКЦИЯ 2

Переменные, типы данных и управление памятью

Память-лента · биты и байты · стек вызовов и куча · значимые и ссылочные типы · IEEE 754

Фоменко М.Ю.

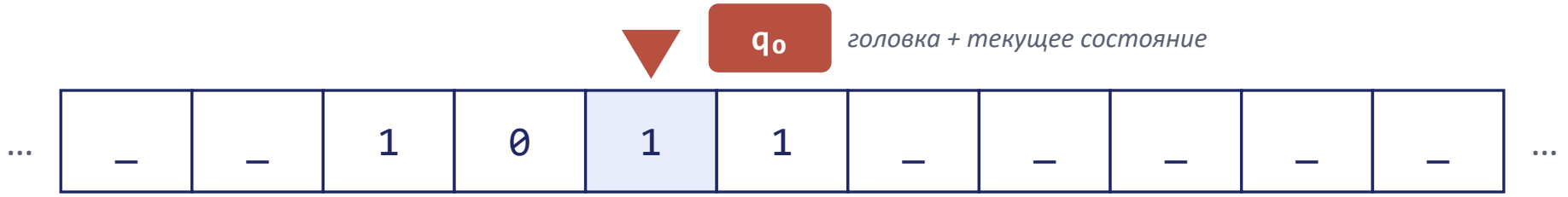
1

Физическая организация памяти

Машина Тьюринга, лента памяти, биты, байты и адреса

Машина Тьюринга: лента, головка, правила

1936 год, Алан Тьюринг — мысленная модель «вычислителя», придуманная ещё до первых компьютеров. Удивительно, но в ней уже есть всё главное, что есть в вашем ноутбуке.



лента уходит в бесконечность в обе стороны

1 Лента

Бесконечная полоса, разбитая на ячейки. В каждой ячейке — один символ (или пусто «_»). Это память.

2 Головка

Смотрит ровно на одну ячейку: читает символ, может записать новый и сдвинуться на 1 влево или вправо.

3 Таблица правил

«В состоянии q, прочитав символ x → запиши y, сдвинься, перейди в состояние q'». Это программа.

Тезис Чёрча–Тьюринга: всё, что вообще можно посчитать алгоритмом, можно посчитать на такой машине. Компьютер принципиально не «умнее» — он просто быстрее.

Мини-пример: прибавить единицу

Число записано в унарном виде: **3 = 111**. Задача — получить 4 = 1111. Программа из двух правил:

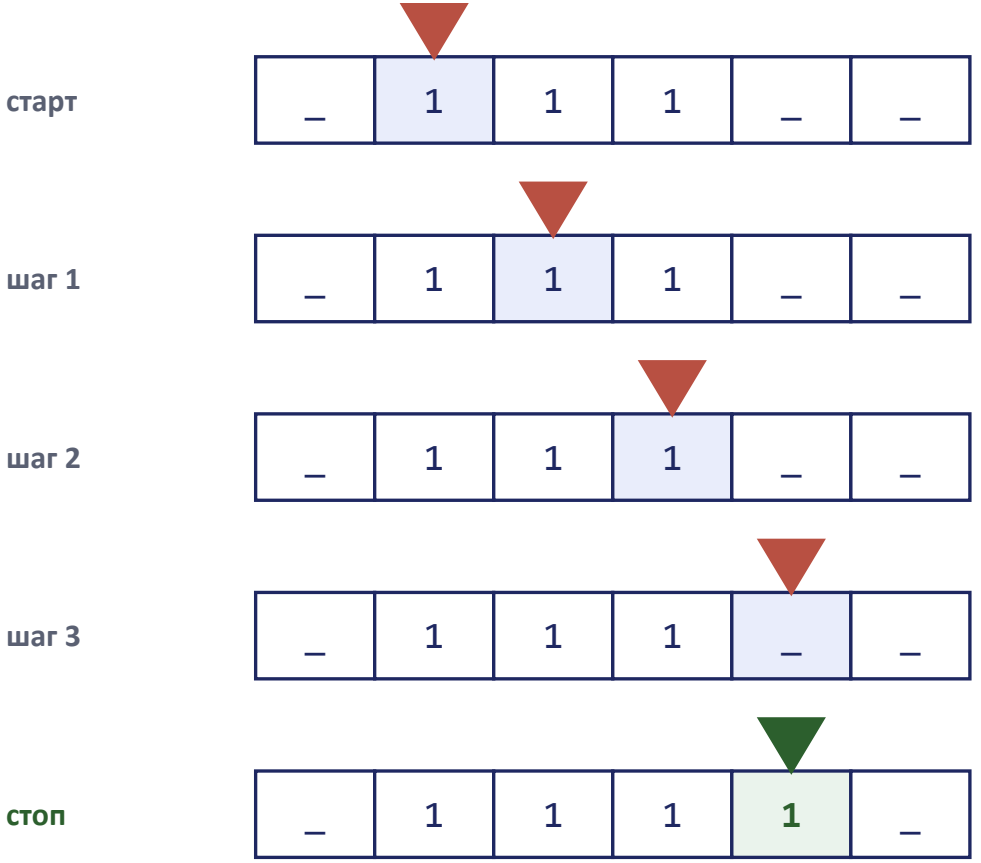
Состояние	Читаем	Пишем	Сдвиг	Дальше
q_0	1	1	→	q_0
q_0	_	1	–	СТОП

По-человечески:

1) пока видишь 1 — оставь её и шагни вправо;

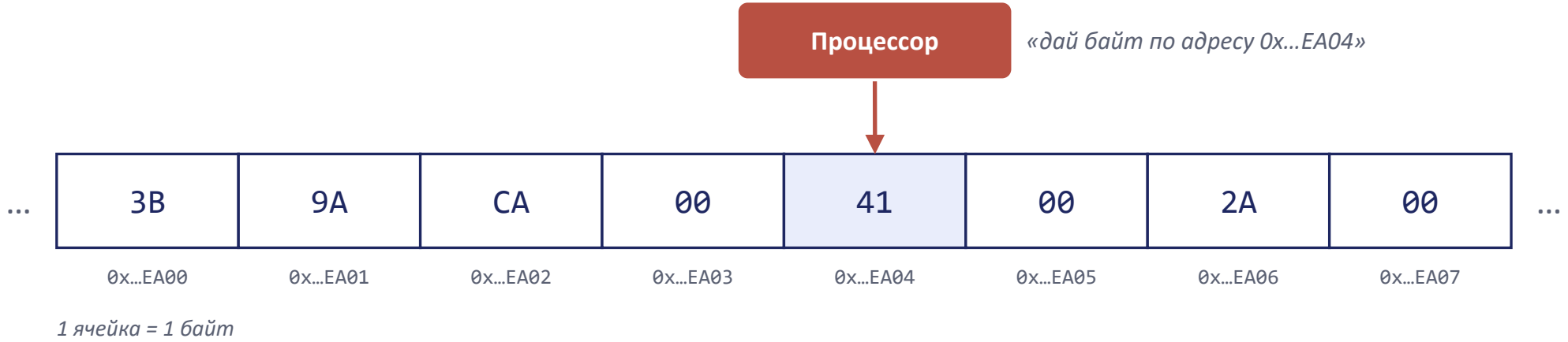
2) увидел пустую ячейку — запиши туда 1 и остановись.

Никакой «магии сложения»: только читать, писать и шагать.



головка ▼ над текущей ячейкой

Память компьютера — та же лента



1 ячейка = 1 байт

Машина Тьюринга	Компьютер (ОЗУ)
Ячейка ленты с символом	Ячейка памяти: 1 байт = 8 бит
Позиция ячейки на ленте	Адрес — номер байта (пишут в 16-ричной: 0x…EA04)
Головка шагает по одной ячейке	Процессор сразу обращается к любому адресу — RAM, Random Access Memory
Лента бесконечна	Память конечна: 8, 16, 32 ГБ

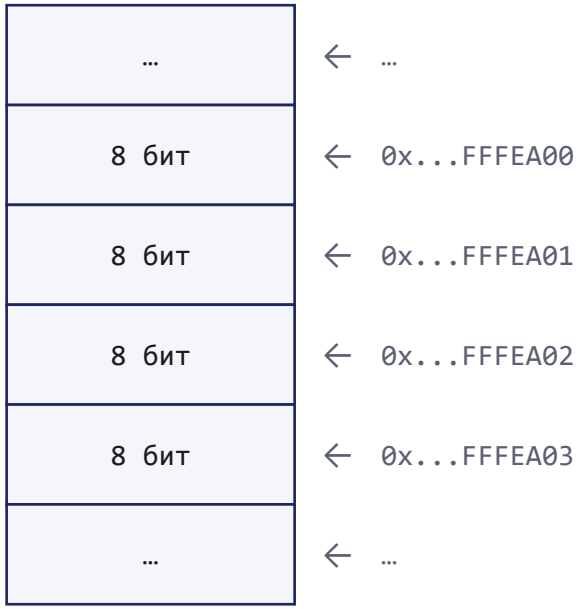
Бит и байт

- Бит — минимальная единица информации, значение 0 или 1
- Байт — 8 бит, обрабатываемые компьютером одновременно



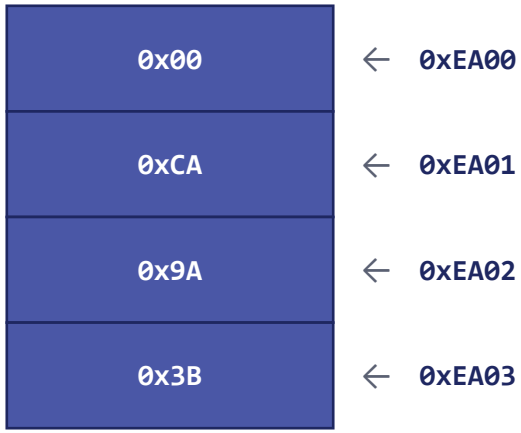
Ячейка памяти и адрес

- Ячейка памяти хранит 1 байт и настраивается на высокий (1) или низкий (0) уровень напряжения
- Адрес памяти — уникальное число, идентифицирующее конкретный байт
- 4 ГБ ОЗУ = 2^{32} байт, адреса от 0 до $2^{32}-1$ (обычно записываются в 16-ричной системе)



Little-Endian и Big-Endian

Записываем число 1 000 000 000 (00111011_10011010_11001010_00000000) начиная с адреса 0xEA00. Куда какой байт?



Little-Endian — так пишут x86 / x86-64 (Intel, AMD)

Little-Endian

Младший байт (little end) — по младшему адресу. Современные ПК: Intel, AMD (x86, x86-64).

Big-Endian

Старший байт (big end) — по младшему адресу. Старые архитектуры: IBM mainframes, PowerPC, SPARC.

2

Данные и типы

Переменная как триада, компиляция и runtime

Переменная — это триада

Переменная — именованная область памяти, в которой хранится значение. Формально это триада: идентификатор (имя), тип и значение, привязанные к области памяти.

```
int userAge;
```

ДЕТАЛИ — НЕ ОБЯЗАТЕЛЬНО

Компиляция

- int → System.Int32
- userAge → идентификатор для доступа к области памяти
- Компилятор генерирует IL с инструкцией выделить 4 байта на стеке

ДЕТАЛИ — НЕ ОБЯЗАТЕЛЬНО

Runtime

- CLR резервирует 4 байта в фрейме стека
- userAge символически связан с адресом этого блока
- Чтение/запись по имени → операция с ячейкой памяти

Суть без деталей: тип переменной определяет, сколько памяти и как её читать — это решается ещё на компиляции.

Тип — контракт и инструкция по интерпретации

C# статически типизирован: тип каждой переменной известен и проверяется на этапе компиляции, а не во время выполнения.

Одна и та же последовательность битов может означать разное — в зависимости от типа:



int → 65

char → 'A'

float / double → часть мантиссы или экспоненты — совсем другое число

Тип — контракт между программистом и компилятором: биты будут интерпретированы предсказуемо, в соответствии с объявленным типом.

3

Стек

LIFO, фреймы, стек вызовов по шагам, StackOverflow

ОРГАНИЗОВАННЫЙ ПОРЯДОК

LIFO и FIFO

LIFO

Last In, First Out — последним пришёл, первым ушёл

Стопка тарелок: моете A, потом B поверх A, потом C поверх B. Возьмёте — сначала C, потом B, потом A.

- Стек вызовов методов
- Ctrl+Z (отмена действия)

FIFO

First In, First Out — первым пришёл, первым ушёл

Очередь в кассу: пришёл A — первый, за ним B, потом C. Обслужат A, потом B, потом C — порядок сохраняется.

- Кнопка «Назад» в браузере
- Очередь печати, обработка запросов

Фрейм стека

Стек — строго структурированная область памяти по принципу LIFO. Управляет исполнением кода на уровне методов: при вызове метода на вершине стека выделяется блок памяти — фрейм стека.

1. **Параметры метода**

   Копии значений аргументов, переданных при вызове

2. **Локальные переменные**

   Память для всех переменных, объявленных внутри метода

3. **Адрес возврата**

   Куда вернуться в коде вызывающего метода после завершения

Push, Pop и характеристики

- push — добавить наверх, pop — снять с вершины; середина стопки недоступна без снятия верхних
- Метод завершается → его фрейм снимается (pop), память освобождается мгновенно
- Управление возвращается по сохранённому адресу возврата

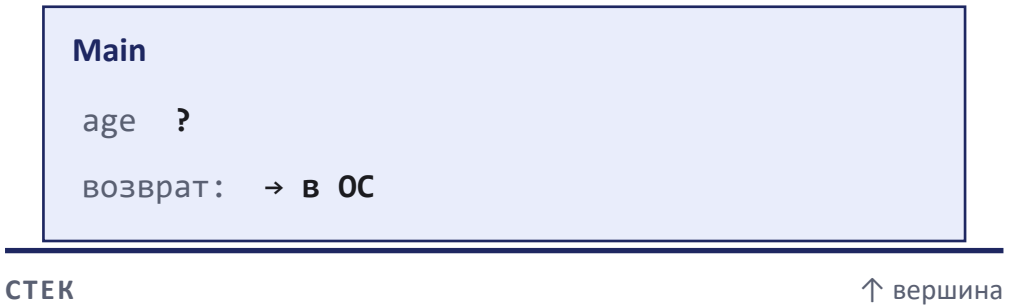
Характеристики

- Жизненный цикл детерминирован: выделение при входе, освобождение при выходе из метода
- Выделение/освобождение — это сдвиг одного регистра (указателя стека) — очень быстро
- Размер фиксирован и невелик (~1-2 МБ на поток) → StackOverflowException при переполнении

Шаг 1. Запуск: на стеке только Main

```
1  static void Main()
2  {
3      int age = ReadAge();
4      Console.WriteLine($"Через год: {age + 1}");
5  }
6
7  static int ReadAge()
8  {
9      Console.Write("Возраст: ");
10     string s = Console.ReadLine();
11     int a = int.Parse(s);
12     return a;
13 }
```

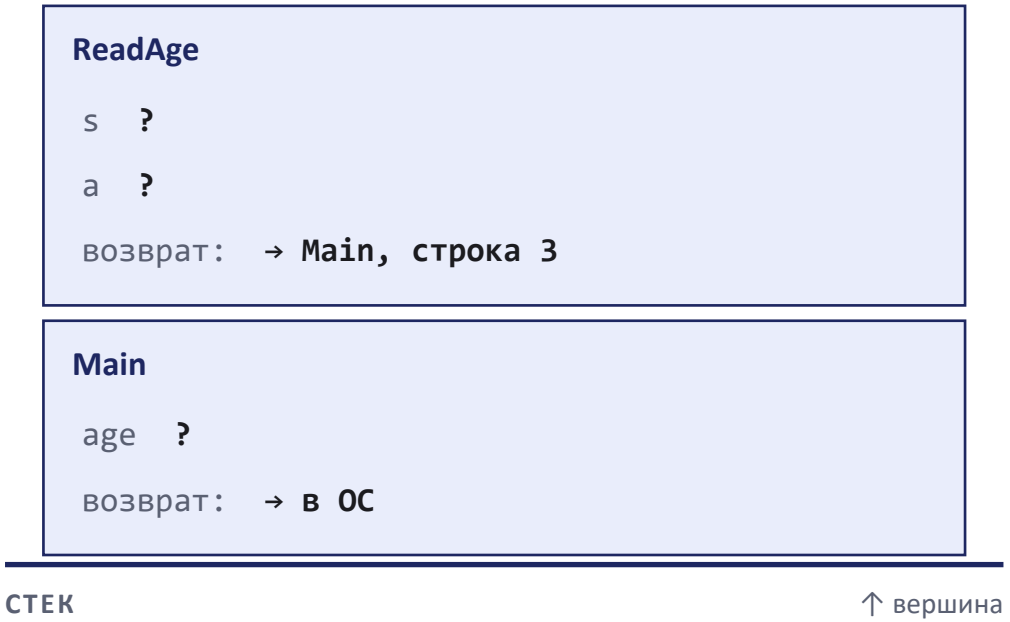
Программа стартует с Main — на стек кладётся его фрейм. Переменная age во фрейме уже есть, но значения ещё нет: сначала нужно вызвать ReadAge().



Шаг 2. Вызов ReadAge(): push

```
 1  static void Main()
 2  {
 3      int age = ReadAge();
 4      Console.WriteLine($"Через год: {age + 1}");
 5  }
 6
 7  static int ReadAge()
 8  {
 9      Console.Write("Возраст: ");
10      string s = Console.ReadLine();
11      int a = int.Parse(s);
12      return a;
13  }
```

Main не закончился — он «на паузе» в строке 3 и ждёт результат. Сверху лёг новый фрейм ReadAge: свои локальные s и a и адрес возврата — куда вернуться потом.

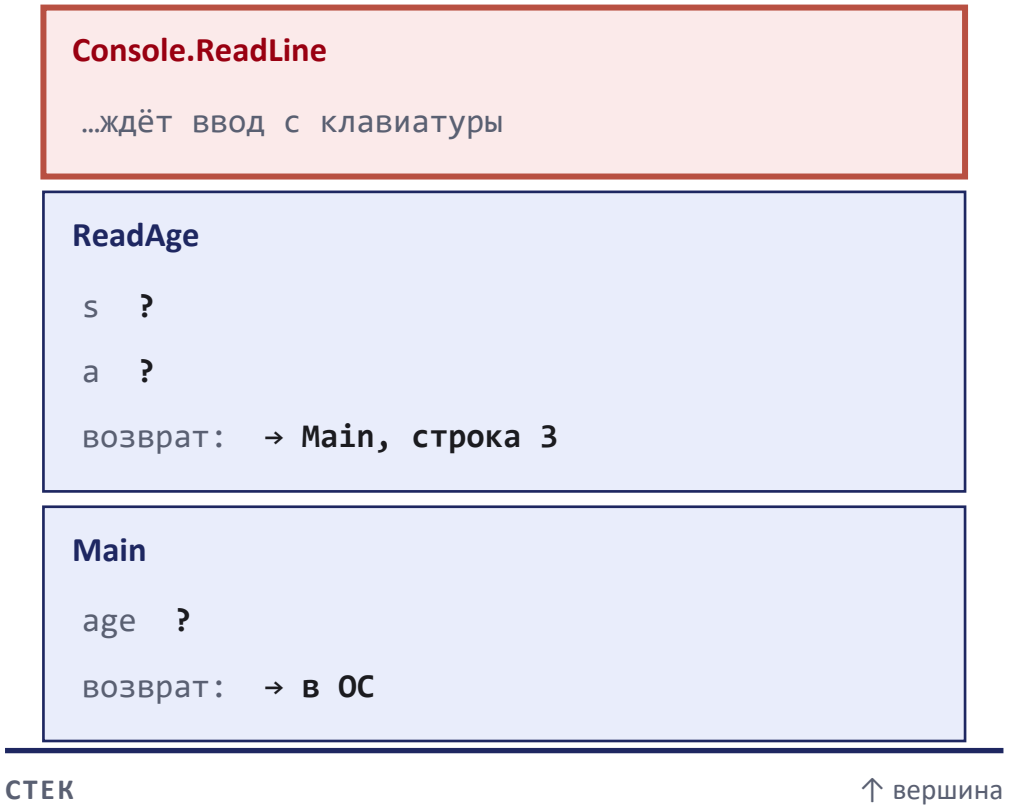


Шаг 3. Console.ReadLine(): программа встала и ждёт

```
1  static void Main()
2  {
3      int age = ReadAge();
4      Console.WriteLine($"Через год: {age + 1}");
5  }
6
7  static int ReadAge()
8  {
9      Console.Write("Возраст: ");
10     string s = Console.ReadLine();
11     int a = int.Parse(s);
12     return a;
13 }
```

⏸ Тут программа встанет и будет ждать

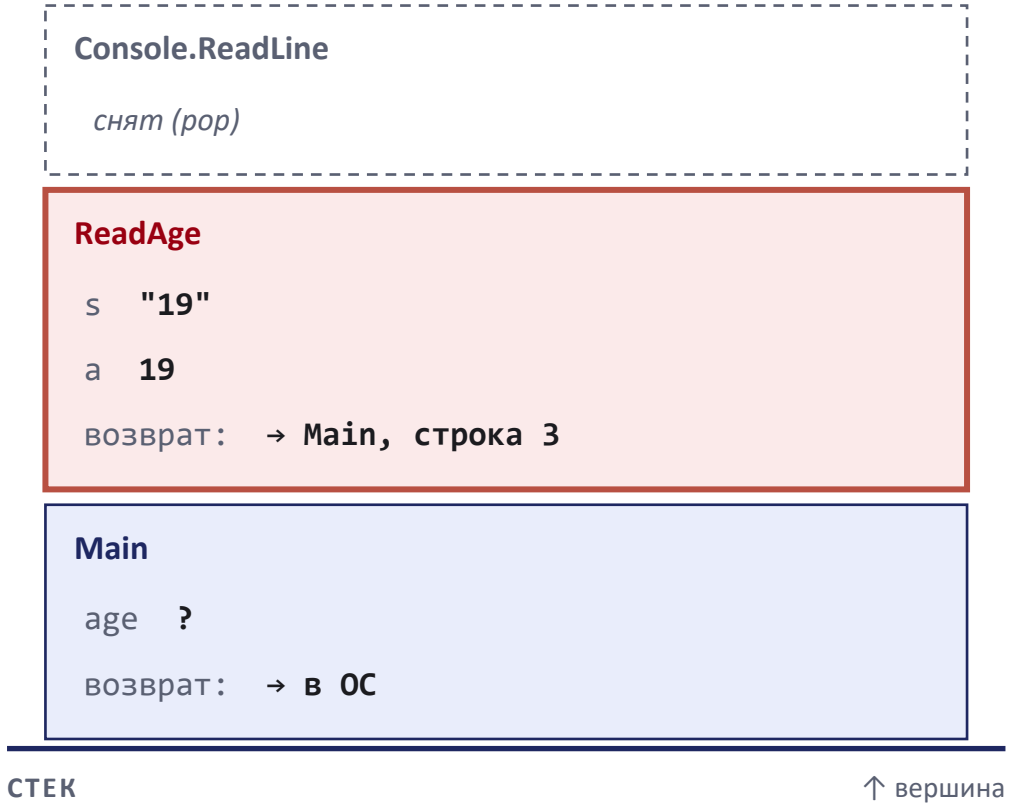
Пока не нажат Enter, дальше строки 10 ничего не выполнится. Все три фрейма живы.



Шаг 4. Ввели «19» + Enter: ReadLine снят (pop)

```
1  static void Main()
2  {
3      int age = ReadAge();
4      Console.WriteLine($"Через год: {age + 1}");
5  }
6
7  static int ReadAge()
8  {
9      Console.Write("Возраст: ");
10     string s = Console.ReadLine();
11     int a = int.Parse(s);
12     return a;
13 }
```

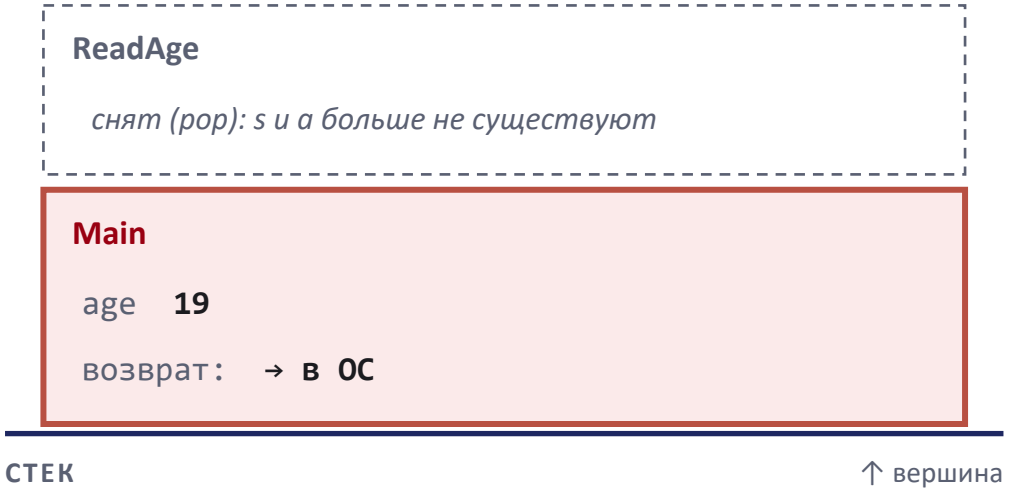
ReadLine вернул строку "19" и его фрейм снят. ReadAge продолжает работу: s = "19", int.Parse даёт a = 19, доходим до return a.



Шаг 5. return a: ReadAge снят, значение — в Main

```
1  static void Main()
2  {
3      int age = ReadAge();
4      Console.WriteLine($"Через год: {age + 1}");
5  }
6
7  static int ReadAge()
8  {
9      Console.Write("Возраст: ");
10     string s = Console.ReadLine();
11     int a = int.Parse(s);
12     return a;
13 }
```

return кладёт 19 «наружу» и снимает фрейм ReadAge — s и a исчезают вместе с ним. По адресу возврата управление попадает в строку 3 Main: age = 19.



Увидеть стек своими глазами

1 **Точка останова**

Встаньте на строку внутри ReadAge и нажмите F9 — слева появится красная точка.

2 **Запуск отладки**

F5. Программа остановится на этой строке, не выполнив её.

3 **Окно «Стек вызовов»**

Отладка → Окна → Стек вызовов (Ctrl+Alt+C). Сверху — текущий метод, ниже — кто его вызвал.

4 **Программа «висит» на ReadLine?**

Отладка → Приостановить все (Ctrl+Alt+Break) — увидите ту же цепочку из шага 3.

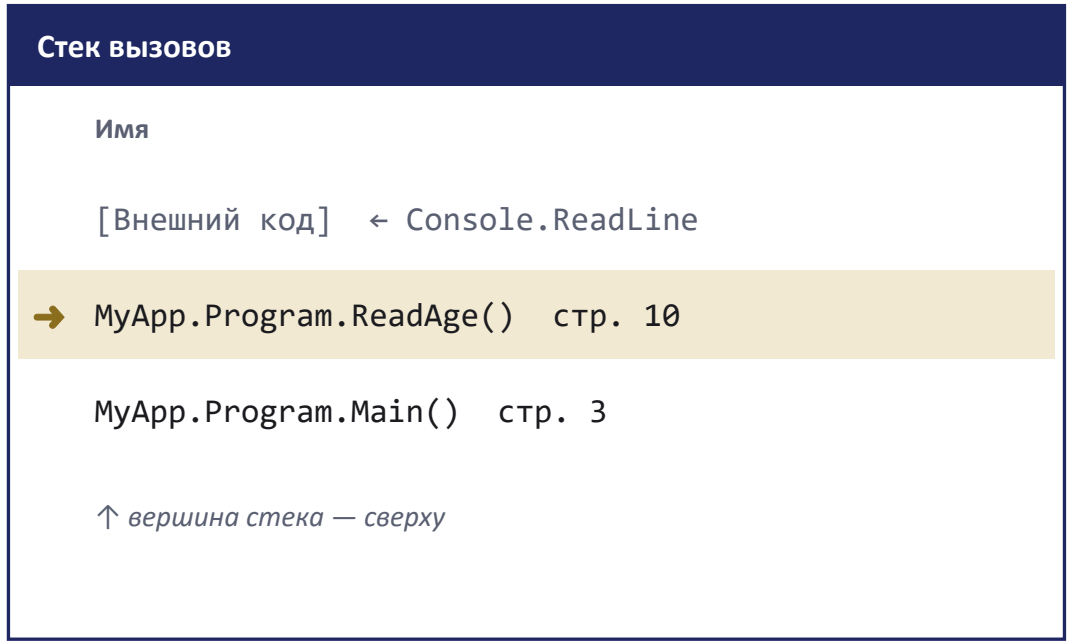


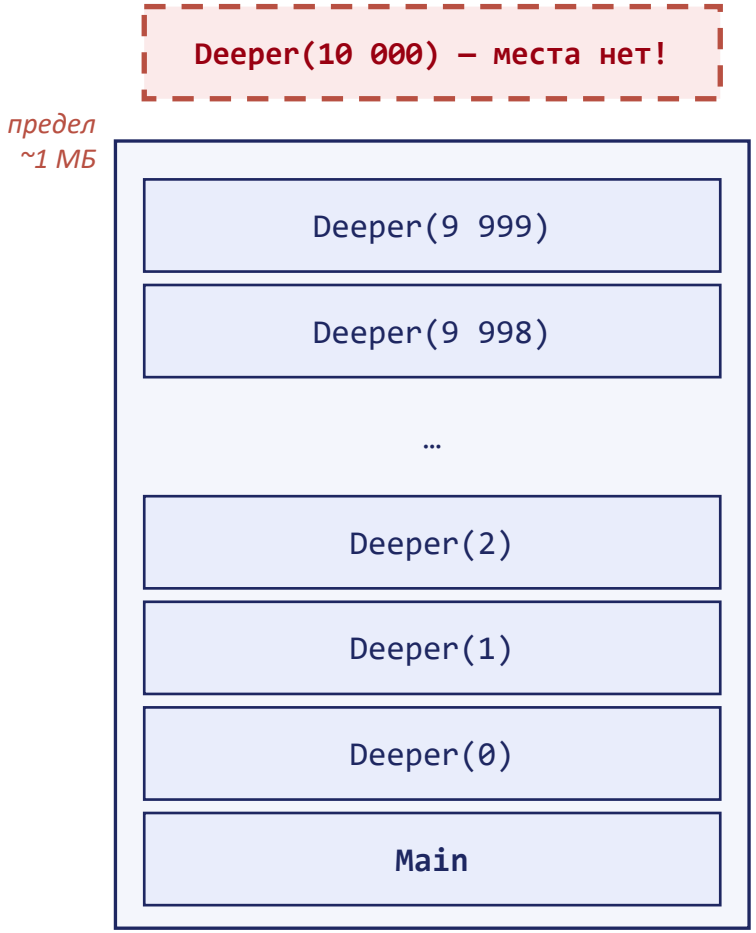
Схема окна (Visual Studio, рус. интерфейс). Имя проекта у вас будет своё.

Это главный инструмент отладки на весь курс: «как я сюда попал?» — смотрите стек вызовов.

Когда стек кончается: StackOverflowException

```
static void Deeper(int n)
{
    Deeper(n + 1); // выхода нет
}
```

- Каждый вызов кладёт на стек новый фрейм, а снять его некому — ни один Deeper не доходит до конца.
- Стек маленький и фиксированный (~1 МБ на поток) → через десятки тысяч вызовов место заканчивается.
- Итог — StackOverflowException: процесс падает целиком, перехватить это исключение нельзя.



Метод, вызывающий сам себя, — рекурсия. Это нормальный приём, если есть условие выхода. Подробнее — позже в курсе.

4

Управляемая куча

new, сборщик мусора, характеристики

Оператор new

Управляемая куча (The Managed Heap) — область памяти CLR для хранения объектов. В отличие от стека, момент освобождения не детерминирован: объект может жить и после завершения создавшего его метода.

1. Найти в куче непрерывный свободный блок нужного размера
2. Инициализировать блок (обнулить)
3. Вызвать конструктор объекта
4. Вернуть адрес (ссылку) на блок — это и станет значением переменной

Сборщик мусора (GC)

- GC периодически приостанавливает программу и «собирает мусор»
- Объект, недостижимый ни по одной ссылке от «корней» — считается мусором
- GC избавляет от ручного освобождения памяти — предотвращает утечки и висячие указатели, характерные для C++

Характеристики кучи

- Объект живёт, пока есть хотя бы одна достижимая ссылка
- Аллокация требует поиска блока — на порядки медленнее стековой
- Освобождение непредсказуемо по времени, решает GC
- Ограничена лишь объёмом ОЗУ и архитектурой ОС

СРАВНЕНИЕ

Стек vs Управляемая куча

Признак	Стек	Управляемая куча
Назначение	Поток исполнения, локальные данные	Объекты с динамическим временем жизни
Структура	LIFO	Неструктурированный пул
Скорость выделения	Очень высокая (сдвиг указателя)	Медленнее (поиск блока)
Время жизни	Ограничено вызовом метода	Пока есть ссылки
Управление	Компилятором и CLR	Сборщиком мусора (GC)
Риск ошибки	StackOverflowException	Паузы GC, фрагментация

5

Значимые и ссылочные типы

Где хранятся данные и что копируется

Value Types: копия значения

Переменная значимого типа хранит значение прямо внутри своей области памяти. При присваивании происходит полное побитовое копирование.

- int, double, float, decimal, byte…
- bool, char
- struct, enum

```
int a = 42;
int b = a;   // копия значения в СТЕКЕ
b = 100;     // 'a' не затронута
// a всё ещё 42
```



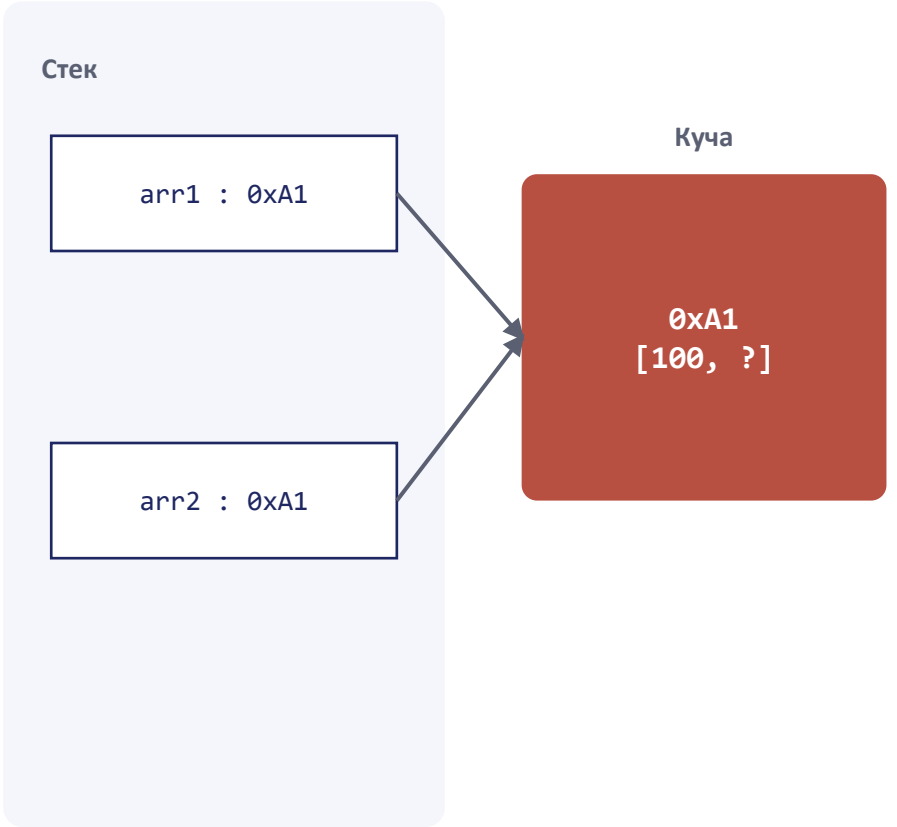
Независимые копии — свои ячейки, свои значения

Reference Types: копия ссылки

Переменная ссылочного типа хранит не сам объект, а ссылку (адрес) на место в куче. Копируется только адрес — не объект.

- Все class, string, все массивы
- delegate, interface, object

```
int[] arr1 = new int[2]; // new: в КУЧЕ
arr1[0] = 42;
int[] arr2 = arr1;  // копия ССЫЛКИ
arr2[0] = 100;
// arr1[0] тоже 100 — общий объект!
```



На ленте: значение и ссылка

```
int a = 42;   int[] arr = { 42, 100 };
```



Значение

Содержимое ячейки и есть данные: в a лежит 42.

Ссылка

Тоже содержимое ячейки — но это число является адресом. «Перейти по ссылке» = прочитать номер и прыгнуть туда.

null

В ячейке «нулевой» адрес: прыгать некуда → NullReferenceException.

null — ссылка в никуда

- null — специальное значение для ссылочных типов: ссылка, не указывающая ни на один объект
- int a = 0 — валидное числовое значение «ноль» (не null!)
- int[] arr = null — переменная существует на стеке, но адреса объекта не содержит

```
int[] arr = null;
arr.Length; // 💥 NullReferenceException
```

NullReferenceException

Одна из самых частых ошибок времени выполнения: среда исполнения пытается перейти по адресу в переменной, но адрес «нулевой» — и никакого объекта там нет.

ИТОГ РАЗДЕЛА

Сводная таблица различий

Характеристика	Значимые типы	Ссылочные типы
Хранится в переменной	Само значение	Адрес объекта в куче
Местоположение	Стек	Куча (данные), стек (ссылка)
Присваивание (=)	Копия значения — 2 независимые копии	Копия ссылки — одна на двоих
Сравнение (==)	Сравнивает значения	Сравнивает ссылки (адреса)
По умолчанию	0, false — не может быть null	null
Примеры	int, bool, struct, enum	string, Array, class, object

Это — минимум, который должен отскакивать от зубов: где живут значимые и ссылочные типы, что копируется при «=», и что такое null.

6

Массивы, строки, вызов функций

Аллокация и передача параметров

Где именно живёт массив

```
// Декларация массива из 100 целых чисел
int[] integerArray = new int[100];
// На стеке выделяется 8 байт (x64) — это ССЫЛКА на область в куче
// В куче выделяется 400 байт данных + 12-24 байта служебной информации
// (длина массива, информация о типе и т.д.)
```

- Сам массив — ссылочный тип: данные элементов лежат в куче, на стеке — только адрес

string: ссылочный, но неизменяемый

- string — ссылочный тип: переменная на стеке хранит адрес объекта в куче
- Объект в куче содержит последовательность символов (char) и служебную информацию
- Но string обладает immutability (неизменяемостью) — это делает поведение похожим на значимые типы в некоторых сценариях

```
string greeting = "Hello";
greeting = greeting + ", World!";

// 1. Новый блок памяти в куче
// 2. Новый строковый объект
// 3. 'greeting' на стеке получает
//    адрес НОВОГО объекта
```

Передача параметров по умолчанию — по значению

В C# по умолчанию — передача по значению (pass-by-value). Но интерпретация зависит от того, значимый это тип или ссылочный.

Значимый тип (int)

Копируется само значение. Метод получает независимую копию — изменения внутри метода не влияют на оригинал.

Ссылочный тип (int[])

Копируется значение ссылки (адрес). Новая локальная ссылка указывает на тот же объект — можно менять его состояние, но не саму исходную ссылку.

Визуальная аналогия

V

Ксерокопия документа

Коллега рисует на своей копии что угодно — ваш оригинал остаётся нетронутым. Это значимый тип.

R

Адрес гаража на бумажке

Две физически разные бумажки, но указывают на один и тот же гараж. Кто то переложил мешки с картошкой — увидите и вы (придя по этому адресу). Это ссылочный тип.

7

Представление float и double

IEEE 754 — компромисс конечных бит и бесконечных чисел

Компромисс: научная запись в двоичном виде

Между 0 и 1 бесконечно много вещественных чисел, а бит — конечное количество (32 для float, 64 для double). Все числа не влезут — стандарт IEEE 754 задаёт компромисс.

```
значение = (-1)^знак × 1.мантисса × 2^(порядок - смещение)
```

🎓 ТОЧНЫЕ ЦИФРЫ — НЕ ОБЯЗАТЕЛЬНО

Тип	Знак	Порядок	Мантисса	Всего
float (Single)	1 бит	8 бит	23 бита	32 бита
double (Double)	1 бит	11 бит	52 бита	64 бита

Отсюда и разница в точности: ~7 значащих цифр у float против ~15–16 у double — это буквально число бит мантиссы.

Почему 0.1 нельзя записать точно

- 1/3 = 0.333… не представимо конечным числом десятичных цифр — так же 0.1 (= 1/10) не представимо конечным числом двоичных цифр
- Двоичная дробь конечна, только если знаменатель — степень двойки (1/2, 1/4, 1/8…)

```
0.1(10) = 0.0001100110011001100110011...(2)  — бесконечная периодическая дробь
```

Компьютер обрезает эту бесконечную запись до 23 (float) или 52 (double) бит мантиссы — 0.1 хранится как ближайшее представимое приближение, а не точное значение.

Следствие: 0.1f + 0.2f != 0.3f

```
float a = 0.1f + 0.2f;
float b = 0.3f;

Console.WriteLine(a == b); // False!
Console.WriteLine(a); // 0.30000001192092896
```

! **Это не баг C#**

Фундаментальное свойство двоичной плавающей точки — проявляется одинаково в C++, Java, Python, JavaScript. Любой язык на IEEE 754.

ФАКУЛЬТАТИВНО

ULP: точность относительная, а не абсолютная

ULP (Unit in the Last Place) — минимальный шаг между соседними представимыми числами в конкретной точке шкалы. Чем больше число, тем крупнее шаг между соседями.

Долгий цикл накопления (например, +0.01f миллион раз): по мере роста суммы растёт и ULP — в какой-то момент 0.01 становится меньше шага сетки и «прибавление» перестаёт быть заметным.

⚠ *Не «переполнение мантиссы» — переполнения не происходит, число не выходит за диапазон. Это потеря относительной точности: приращение не помещается в доступную разрядность мантиссы на данном масштабе.*

double (52 бита мантиссы) накапливает погрешность медленнее float (23 бита), но подвержен той же проблеме.

А для денег — decimal

- decimal (128 бит) — конечное десятичное представление, а не двоичное; жертвует диапазоном и скоростью ради точности в десятичной арифметике

Итог

- float/double — компромисс: конечные биты на бесконечное множество вещественных чисел
- «Простые» десятичные дроби чаще всего не представимы в двоичном виде точно
- Точность относительная (ULP растёт вместе со значением) — длинные накопления теряют её быстрее, чем кажется
- Деньги и точные дроби — decimal

Спасибо за внимание

Лабораторная работа №1, задание 4 «Иллюзия точности» — закрепим эту тему на практике